

IA-REFLEXION-C: Antes de usar la IA pensábamos que para obtener el "top 5" tendríamos que ordenar todo el diccionario, lo cual toma tiempo. Aprendimos que podemos combinar estructuras: el dict nos da búsquedas instantáneas gracias a su función hash (que actúa como un "ticket directo" al dato), y usar en paralelo una Cola de Prioridad (Heap) nos permite mantener los 5 mejores elementos siempre filtrados en la cima, resolviendo el problema de forma mucho más eficiente.

"""

streamMX_hash_s12.py

Semana 12 · Tablas Hash: Conceptos y Uso

Equipo: [NOMBRE DEL EQUIPO]

Integrantes: [LISTA]

Fecha: [FECHA]

COMPRENDE-1: ¿Qué hace una función hash? (1–2 oraciones)

Toma un dato de entrada (como una cadena de texto) y lo convierte matemáticamente en un número entero, el cual sirve como el índice exacto en la memoria donde se guardará ese dato.

#

COMPRENDE-2: ¿Qué es el factor de carga y por qué importa?

Es la proporción entre los elementos guardados y el tamaño total de la tabla (qué tan llena está). Importa porque si es muy alto, aumentan las "colisiones" (dos datos que intentan ocupar el mismo índice) y la tabla pierde su velocidad instantánea.

#

COMPRENDE-3: ¿En qué caso el dict de Python deja de ser suficiente?

Deja de ser suficiente cuando necesitamos buscar datos por rangos (ej. puntajes del 5 al 7) o mantener un orden específico (como un "Top 5"), ya que los diccionarios no garantizan un orden de valores.

#

IA-REFLEXION-C: Antes de usar la IA pensábamos que para obtener el "top 5" tendríamos que ordenar todo el diccionario, lo cual toma tiempo. Aprendimos que podemos combinar estructuras: el dict nos da búsquedas instantáneas, y usar en paralelo una Cola de Prioridad (Heap) nos permite mantener los 5 mejores elementos siempre filtrados en la cima de forma eficiente.

"""

import time

import bisect

#

DATOS DEL CATÁLOGO STREAMMX

Formato: "ID" → (titulo, puntaje_float, genero)

#

catalogo_base = {

```

"MX-001": ("Interstellar",      8.2, "Sci-Fi"),
"MX-002": ("El Laberinto del Fauno", 7.5, "Fantasía"),
"MX-003": ("Origen",          9.1, "Sci-Fi"),
"MX-004": ("Dune Parte 1",      6.8, "Sci-Fi"),
"MX-005": ("Oppenheimer",      8.7, "Drama"),
"MX-006": ("La Señal",         7.5, "Thriller"),
"MX-007": ("Everything Everywhere", 9.4, "Comedia"),
"MX-008": ("Los Elegidos",      6.2, "Drama"),
"MX-009": ("The Batman",       8.0, "Acción"),
"MX-010": ("Gravedad",         7.5, "Sci-Fi"),
"MX-011": ("Parasite",         9.0, "Thriller"),
"MX-012": ("Midsommar",        7.1, "Terror"),
}

```

#

```

# PATRÓN 1 · LOOKUP DIRECTO — Catálogo dinámico (R1)
# Resuelve el Dilema Final de S11: inserción O(1) sin re-ordenar
#

```

```

def buscar_pelicula(catalogo: dict, id_pelicula: str):
    """

```

Retorna la tupla (titulo, puntaje, genero) si el ID existe.
Retorna None sin lanzar KeyError si no existe.

```

    # DECISIÓN: Usamos .get() porque si la clave no existe retorna None limpiamente.
    # Si usáramos catalogo[id_pelicula], el programa arrojaría un error fatal (KeyError) si el ID
    no se encuentra.
    """

```

```

    return catalogo.get(id_pelicula)

```

```

def agregar_pelicula(catalogo: dict, id_pelicula: str, datos: tuple) -> None:
    """

```

Agrega o actualiza una película en el catálogo.
Si el ID ya existe, actualiza los datos (no duplica).

```

    # DECISIÓN: En un dict la inserción es O(1) (tiempo constante, un solo paso) porque la
    función hash nos da el lugar exacto.
    # En una lista ordenada era O(n) porque, al insertar, teníamos que recorrer y desplazar
    todos los elementos siguientes para hacerle espacio.
    """

```

```

    catalogo[id_pelicula] = datos

```

#

PATRÓN 2 · CONTEO DE FRECUENCIAS — Estadísticas por género (R2)

#

```
def contar_por_genero(catalogo: dict) -> dict:
```

```
    """
```

```
    Retorna un dict {genero: cantidad_de_peliculas}.
```

```
    Ejemplo: {"Sci-Fi": 4, "Drama": 2, "Thriller": 2, ...}
```

```
    # DECISIÓN: Usar .get(genero, 0) pide el valor actual, o asume 0 si es la primera vez que
    vemos ese género.
```

```
    # Esto nos ahorra escribir un bloque 'if/else' con el operador 'in'.
```

```
    """
```

```
    conteos = {}
```

```
    for id_pelicula, datos in catalogo.items():
```

```
        titulo, puntaje, genero = datos
```

```
        conteos[genero] = conteos.get(genero, 0) + 1
```

```
    return conteos
```

#

PATRÓN 3 · AGRUPAMIENTO / BUCKET — Películas por rango de puntaje (R3)

#

```
def agrupar_por_rango(catalogo: dict) -> dict:
```

```
    """
```

```
    Agrupa películas en tres rangos de puntaje:
```

```
    "popular" → puntaje en [7.0, 8.0)
```

```
    "destacado" → puntaje en [8.0, 9.0)
```

```
    "premium" → puntaje en [9.0, 10.0]
```

```
    Retorna: {"popular": [...], "destacado": [...], "premium": [...]}
```

```
    Cada lista contiene tuplas (id, titulo, puntaje).
```

```
    # DECISIÓN: Inicializamos el dict antes del loop para asegurar que las listas de las tres
    categorías existan desde el principio,
```

```
    # evitando errores si alguna categoría termina sin películas.
```

```
    """
```

```
    grupos = {"popular": [], "destacado": [], "premium": []}
```

```
    for id_pelicula, datos in catalogo.items():
```

```

    titulo, puntaje, genero = datos
    if 7.0 <= puntaje < 8.0:
        grupos["popular"].append((id_pelicula, titulo, puntaje))
    elif 8.0 <= puntaje < 9.0:
        grupos["destacado"].append((id_pelicula, titulo, puntaje))
    elif 9.0 <= puntaje <= 10.0:
        grupos["premium"].append((id_pelicula, titulo, puntaje))
    return grupos

```

```

def peliculas_en_rango(grupos: dict, rango: str) -> list:

```

```

    """

```

```

    Retorna la lista de películas en el rango dado.

```

```

    Retorna lista vacía si el rango no existe (sin KeyError).

```

```

    # DECISIÓN: Usamos .get() asignándole una lista vacía [] como valor por defecto,

```

```

    # así retornará [] si alguien pide un rango que no existe (ej. "malo").

```

```

    """

```

```

    return grupos.get(rango, [])

```

```

#

```

```

# PATRÓN 4A · DEDUPLICACIÓN — Detectar títulos duplicados (R4)

```

```

#

```

```

def procesar_lote(catalogo: dict, lote: list) -> dict:

```

```

    """

```

```

    Recibe una lista de tuplas (id, titulo, puntaje, genero).

```

```

    Inserta en el catálogo solo las que NO estén ya por título.

```

```

    Retorna: {"nuevas": int, "duplicadas": int}

```

```

    # DECISIÓN: Usamos un 'set' (conjunto) de Python porque, internamente, usa la misma
    arquitectura de tabla hash que un dict.

```

```

    # Esto permite saber si un título existe al instante (O(1)), mucho más rápido que buscar
    en una lista (O(n)).

```

```

    """

```

```

    titulos_existentes = {datos[0] for datos in catalogo.values()} # set de títulos

```

```

    resultado = {"nuevas": 0, "duplicadas": 0}

```

```

    for id_p, titulo, puntaje, genero in lote:

```

```

        if titulo not in titulos_existentes:

```

```

            catalogo[id_p] = (titulo, puntaje, genero)

```

```

            titulos_existentes.add(titulo) # Lo agregamos al set para futuros chequeos del lote

```

```

            resultado["nuevas"] += 1

```

```

    else:
        resultado["duplicadas"] += 1

return resultado

#


---




---


# PATRÓN 4B · CACHÉ / MEMOIZACIÓN — Recomendaciones por usuario (R5)
#


---




---



_cache_recomendaciones = {} # dict global — persiste entre llamadas

def _generar_recomendaciones(user_id: str, catalogo: dict) -> list:
    """
    Simula un motor costoso: tarda ~100ms por llamada.
    NO modificar esta función.
    """
    time.sleep(0.1) # simula latencia del motor
    peliculas = list(catalogo.values())
    peliculas.sort(key=lambda x: x[1], reverse=True)
    return peliculas[:5]

def obtener_recomendaciones(user_id: str, catalogo: dict) -> list:
    """
    Primera llamada: genera recomendaciones (tarda ~100ms) y las guarda en caché.
    Llamadas siguientes: retorna desde caché en O(1) — sin llamar a
    _generar_recomendaciones.

    # DECISIÓN: El user_id es una cadena única e inmutable (como un número de cuenta),
    # lo que lo hace perfecto para ser hashado y usado como clave directa en un
    diccionario.
    """
    if user_id in _cache_recomendaciones:
        return _cache_recomendaciones[user_id]

    # Si no está en caché, generamos, guardamos y retornamos
    recomendaciones = _generar_recomendaciones(user_id, catalogo)
    _cache_recomendaciones[user_id] = recomendaciones
    return recomendaciones

```

#

BENCHMARK — Pipeline S11 (lista + bisect) vs dict

#

```
def benchmark_comparativo(n_valores: list = None):
```

```
    """
```

```
        Compara el tiempo de búsqueda entre:
```

- Pipeline S11: lista de IDs ordenados + bisect.bisect_left
- Dict S12: acceso directo por clave

```
        # REFLEXIONA-1: La diferencia de tiempos ocurre porque la búsqueda binaria (bisect) es O(log n), es decir, divide a la mitad repetidamente. El diccionario es O(1), va directo usando matemáticas.
```

```
        # REFLEXIONA-2: El dict mantiene su ventaja con n grandes porque su tiempo de acceso no depende de cuántos elementos haya en la tabla; la fórmula matemática toma el mismo tiempo en calcular el índice para 10 que para un millón.
```

```
        # REFLEXIONA-3: El pipeline S11 sería preferible si necesitamos hacer búsquedas por rango (por ejemplo, buscar todos los IDs entre MX-005 y MX-020) o si necesitamos extraer los datos de forma ordenada constantemente, algo que el dict no puede hacer sin usar algoritmos de ordenamiento extras.
```

```
    """
```

```
    if n_valores is None:
```

```
        n_valores = [10_000, 100_000, 1_000_000]
```

```
    print(f'{n}>12} | {'Lista+bisect (ms)':>20} | {'Dict (ms)':>15} | {'Speedup':>10}")
```

```
    print("-" * 65)
```

```
    for n in n_valores:
```

```
        claves = [f"ID-{i:07d}" for i in range(n)]
```

```
        valor_dummy = ("Película", 8.0, "Drama")
```

```
        # Pipeline S11: lista ordenada + bisect
```

```
        lista_ordenada = sorted(claves)
```

```
        t0 = time.perf_counter()
```

```
        for _ in range(1000):
```

```
            objetivo = f"ID-{n//2:07d}"
```

```
            idx = bisect.bisect_left(lista_ordenada, objetivo)
```

```
            _ = lista_ordenada[idx] if idx < len(lista_ordenada) else None
```

```
        t_lista = (time.perf_counter() - t0) * 1000
```

```
        # Dict S12: acceso directo
```

```
        d = {k: valor_dummy for k in claves}
```

```
        t0 = time.perf_counter()
```

```
        for _ in range(1000):
```

```

        _ = d.get(f"ID-{n//2:07d}")
        t_dict = (time.perf_counter() - t0) * 1000

        speedup = t_lista / t_dict if t_dict > 0 else float('inf')
        print(f"{n:>12,} | {t_lista:>20.3f} | {t_dict:>15.3f} | {speedup:>10.1f}×")

#


---




---


# BLOQUE PRINCIPAL — Casos de demostración y benchmark
#


---




---



if __name__ == "__main__":
    import copy
    cat = copy.deepcopy(catalogo_base)

    print("=" * 60)
    print("PATRÓN 1 · Lookup Directo")
    print("=" * 60)
    print("MX-007:", buscar_pelicula(cat, "MX-007"))
    print("MX-999:", buscar_pelicula(cat, "MX-999"))
    agregar_pelicula(cat, "MX-013", ("Dune Parte 2", 8.5, "Sci-Fi"))
    print("MX-013 (nueva):", buscar_pelicula(cat, "MX-013"))

    print("\n" + "=" * 60)
    print("PATRÓN 2 · Conteo por Género")
    print("=" * 60)
    conteos = contar_por_genero(catalogo_base)
    print("Conteos:", conteos)
    print("Sci-Fi:", conteos.get("Sci-Fi"), "→ esperado: 4")
    print("Drama:", conteos.get("Drama"), "→ esperado: 2")

    print("\n" + "=" * 60)
    print("PATRÓN 3 · Agrupamiento por Rango")
    print("=" * 60)
    grupos = agrupar_por_rango(catalogo_base)
    for rango, peliculas in grupos.items():
        print(f" {rango}: {len(peliculas)} películas")
    print("Premium (9–10):", len(peliculas_en_rango(grupos, "premium")), "→ esperado: 3")
    print("Rango inexistente:", peliculas_en_rango(grupos, "epico"), "→ esperado: []")

    print("\n" + "=" * 60)
    print("PATRÓN 4A · Deduplicación")
    print("=" * 60)
    lote_con_duplicados = [

```

```

("MX-013", "Interestelar", 8.2, "Sci-Fi"),
("MX-014", "Avatar: El Camino del Agua", 7.6, "Acción"),
("MX-015", "Parasite", 9.0, "Thriller"),
("MX-016", "Tár", 7.8, "Drama"),
]
resultado = procesar_lote(copy.deepcopy(catalogo_base), lote_con_duplicados)
print("Resultado:", resultado, "→ esperado: {'nuevas': 2, 'duplicadas': 2}")

print("\n" + "=" * 60)
print("PATRÓN 4B · Caché de Recomendaciones")
print("=" * 60)
t0 = time.perf_counter()
r1 = obtener_recomendaciones("user_42", catalogo_base)
t1 = time.perf_counter() - t0
t0 = time.perf_counter()
r2 = obtener_recomendaciones("user_42", catalogo_base)
t2 = time.perf_counter() - t0
print(f"1er llamado: {t1*1000:.1f}ms")
print(f"2do llamado: {t2*1000:.1f}ms")
if t1 > 0 and t2 > 0:
    print(f"Speedup caché: {t1/t2:.0f}× → esperado: ≥ 100×")

print("\n" + "=" * 60)
print("BENCHMARK — Pipeline S11 vs Dict S12")
print("=" * 60)
benchmark_comparativo()

```

——— Evidencia embebida al final del archivo —————

ESTRATEGIA: Nos enfocamos en aprovechar al máximo las operaciones $O(1)$ de Python. Sustituimos estructuras lineales por diccionarios y sets (`.get`, memoización y verificación de existencia rápida), priorizando evitar errores como `KeyError` mediante métodos seguros.

VALIDA: Los casos T1, T2, T3, T5 y T6 pasaron al primer intento sin problemas. El caso T4 requirió corrección porque habíamos clasificado incorrectamente el límite exacto de 8.0. El error ocurrió porque usamos la condición `<= 8.0` para el grupo "popular", lo que "robaba" las películas que debían ir en "destacado"; lo solucionamos ajustando estrictamente los rangos a `< 8.0` y `>= 8.0`.

IA-REFLEXION-A: Aprendimos que el uso de `.get()` es vital en el manejo seguro de estructuras dinámicas. Además, consolidamos la diferencia real entre el acceso directo matemático de una tabla hash contra los saltos lógicos de la búsqueda binaria.

IA-REFLEXION-R: Antes creíamos que un diccionario era la solución definitiva para todo porque es muy rápido. Al usar la IA para analizar los límites (D5), corregimos nuestro razonamiento y entendimos que las estructuras no compiten, sino que se complementan: el dict es invencible para buscar un ID exacto, pero una estructura ordenada (como la lista de S11 o un Heap) sigue siendo estrictamente necesaria cuando nos importa el orden o los rangos.

REFLEXIONA-1: La diferencia de tiempos ocurre porque la búsqueda binaria de la lista (S11) necesita dividir el espacio a la mitad repetidamente (tiempo $O(\log n)$), mientras que el diccionario calcula la dirección de memoria exacta a través de una función hash y accede en un solo paso (tiempo $O(1)$).

REFLEXIONA-2: El dict mantiene su gran ventaja conforme 'n' crece porque su tiempo de acceso es constante ($O(1)$). Calcular el índice matemático toma exactamente el mismo esfuerzo computacional para una tabla con 10 elementos que para una con un millón.

REFLEXIONA-3: El pipeline de S11 (lista ordenada) sería PREFERIBLE al dict si StreamMX necesitara mostrar el catálogo completo en orden alfabético constantemente, o si los usuarios hicieran búsquedas frecuentes por rangos de IDs, cosas que un diccionario no puede hacer eficientemente.

IA-REFLEXION-V: Durante la validación encontramos un "caso límite" (una situación donde los datos están exactamente en la frontera de dos condiciones). Específicamente, descubrimos que al evaluar datos de punto flotante (números decimales), un error mínimo en los operadores relacionales (usar $\leq$ en lugar de $<$) hace que los datos caigan en categorías equivocadas. La IA nos ayudó a visualizar que debemos ser matemáticamente estrictos en las fronteras de los rangos para que ninguna película quede fuera o duplicada en los grupos.

IA-REFLEXION-P: Para la Semana 13 anticipamos que el siguiente paso lógico es estudiar los Árboles Binarios de Búsqueda Balanceados (como los árboles AVL o Red-Black). Aprendimos que el gran límite del dict es su incapacidad para mantener orden y hacer búsquedas por rangos. Un árbol balanceado resolverá este problema de StreamMX al darnos lo mejor de ambos mundos: inserciones dinámicas rápidas en $O(\log n)$ y la capacidad de extraer rangos de puntajes ordenados sin tener que iterar todo el catálogo.